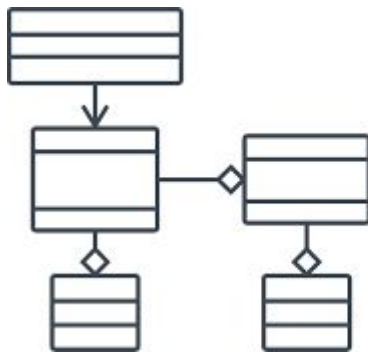


POO: Conceitos

ESCOLA ESTADUAL DE ENSINO PROFISSIONAL
Prof. DAVI MAGALHÃES
PROGRAMAÇÃO ORIENTADA A OBJETOS
AULA 01

Programação Orientada a Objetos

Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o código em torno de "objetos", que são instâncias de classes. Esses objetos encapsulam dados (atributos) e comportamentos (métodos) relacionados.



POO

1 É a Programação Orientada a Objetos e consiste num paradigma de linguagem de programação.

POO: encapsulamento

Encapsulamento: Agrupa dados e métodos que operam sobre esses dados dentro de uma única unidade (classe), escondendo detalhes internos e expondo apenas o necessário.

encapsulamento

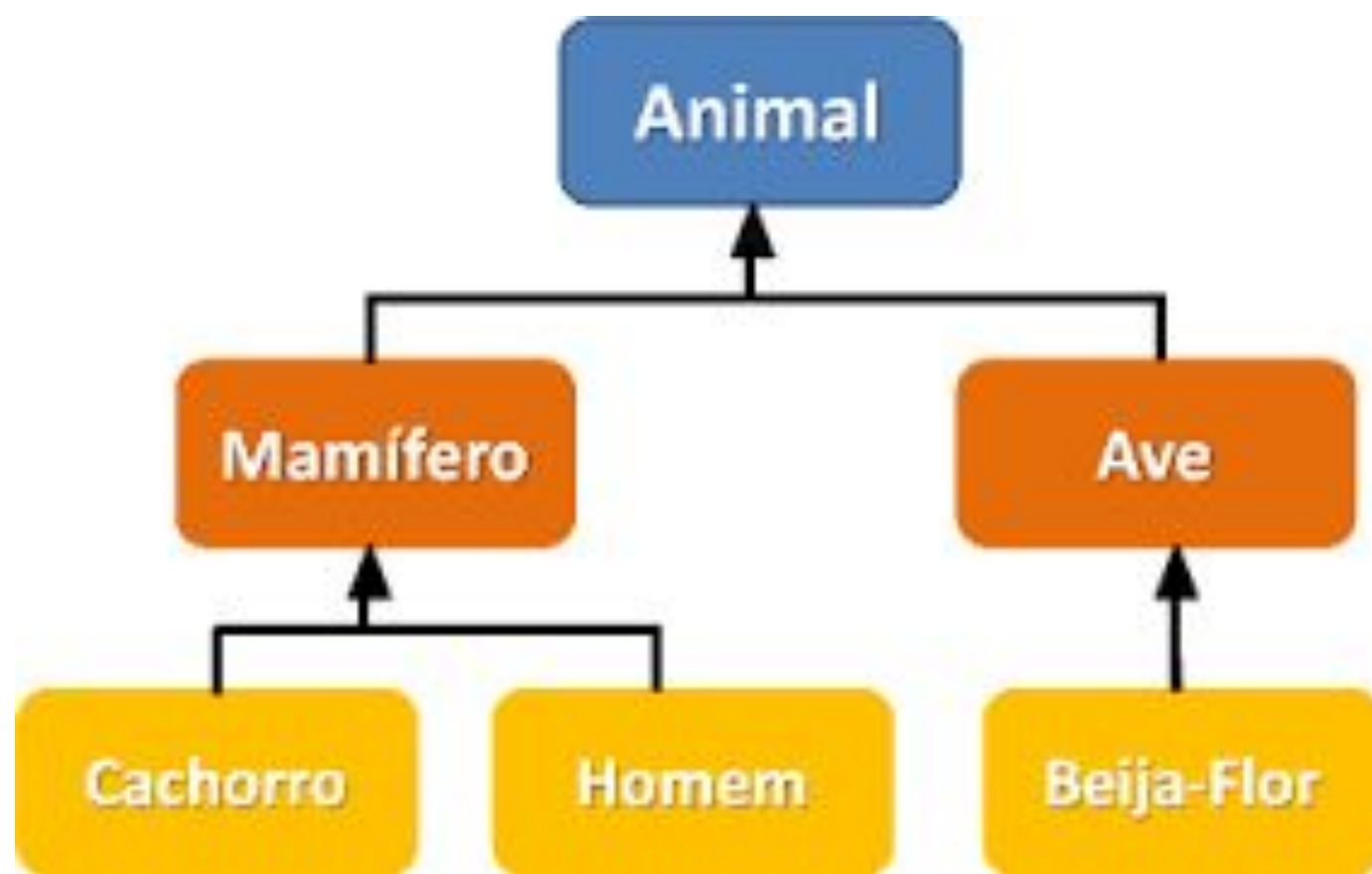
nome masculino

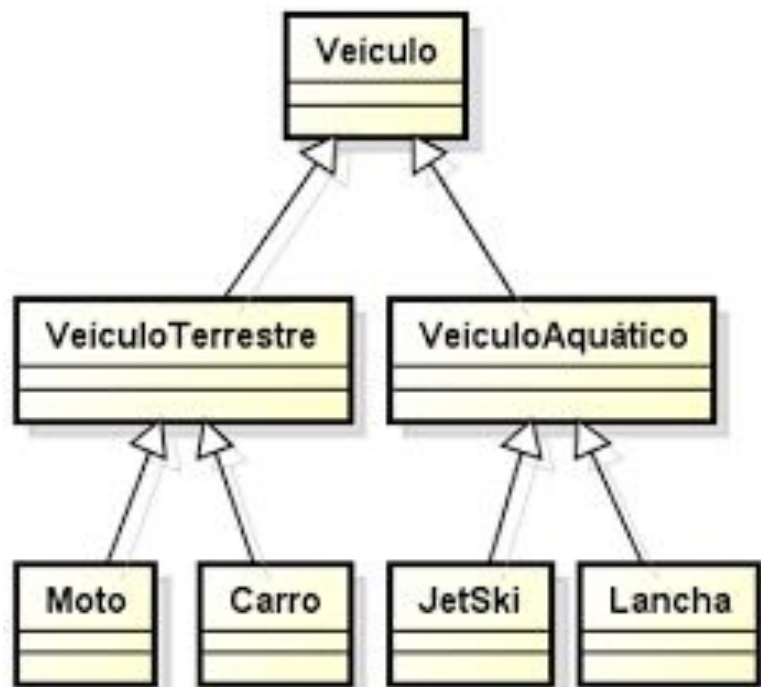
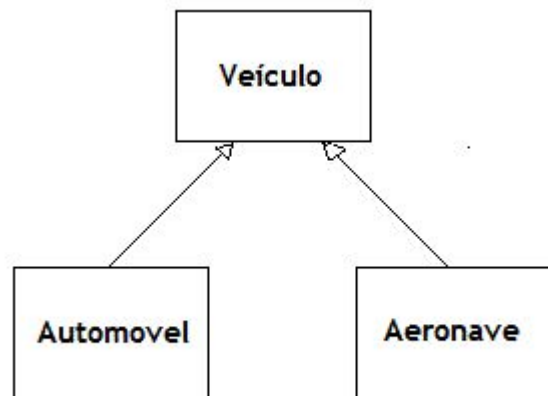
1. ato ou efeito de encapsular, de encerrar em cápsula ou de fechar com cápsula
2. *figurado* isolamento; separação
3. *figurado* contenção



POO: herança

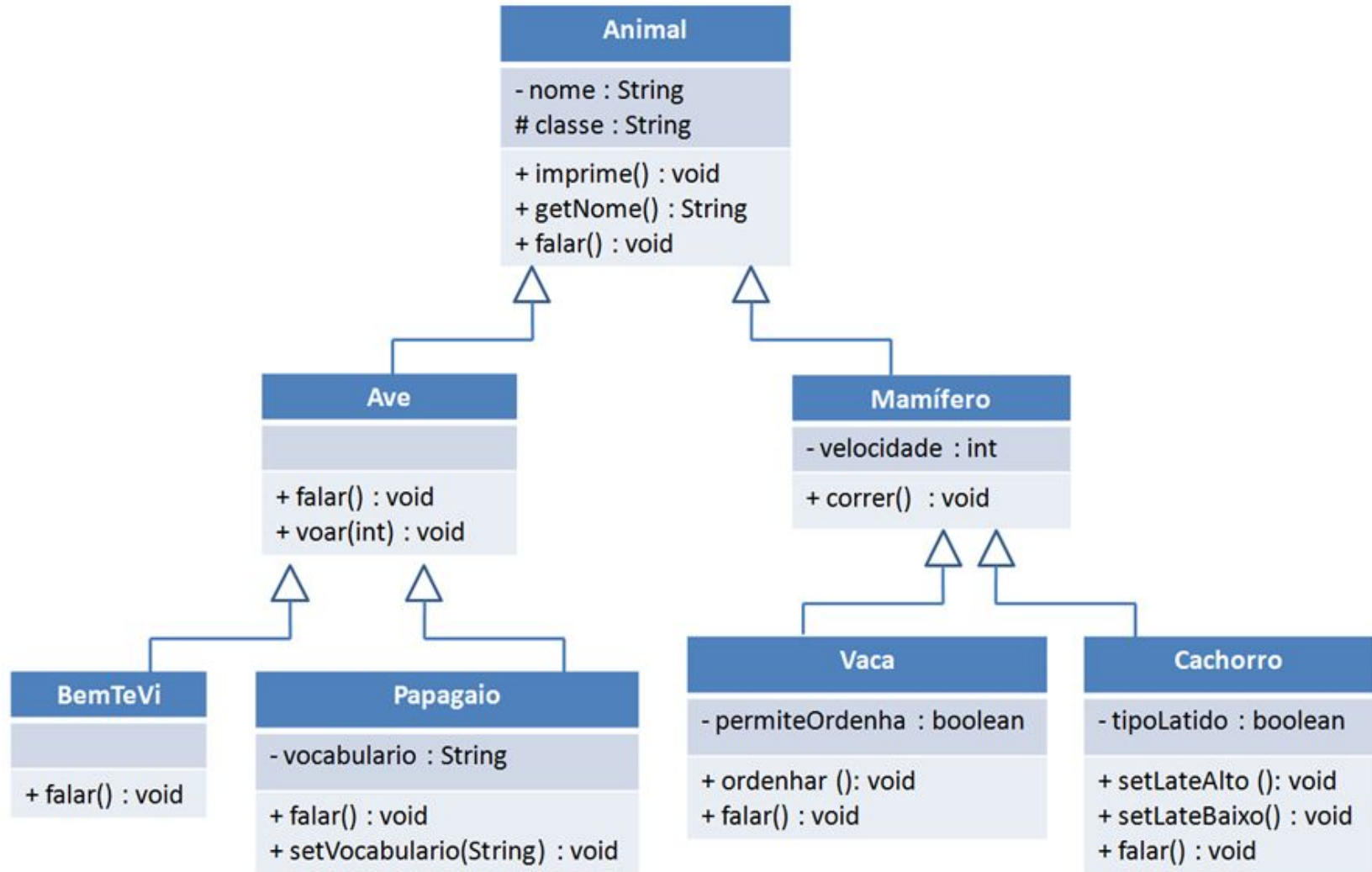
Herança: Permite que uma classe (subclasse) herde propriedades e comportamentos de outra classe (superclasse), promovendo reutilização de código.





POO: polimorfismo

Polimorfismo: Permite que objetos de diferentes classes sejam tratados de maneira uniforme, mesmo que executem comportamentos específicos de maneira distinta.

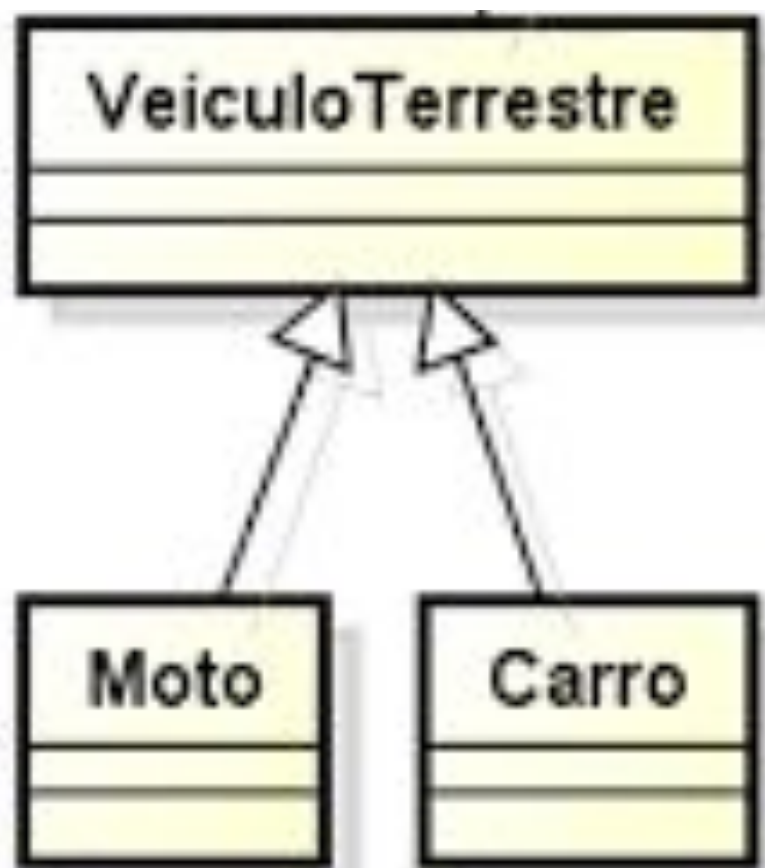


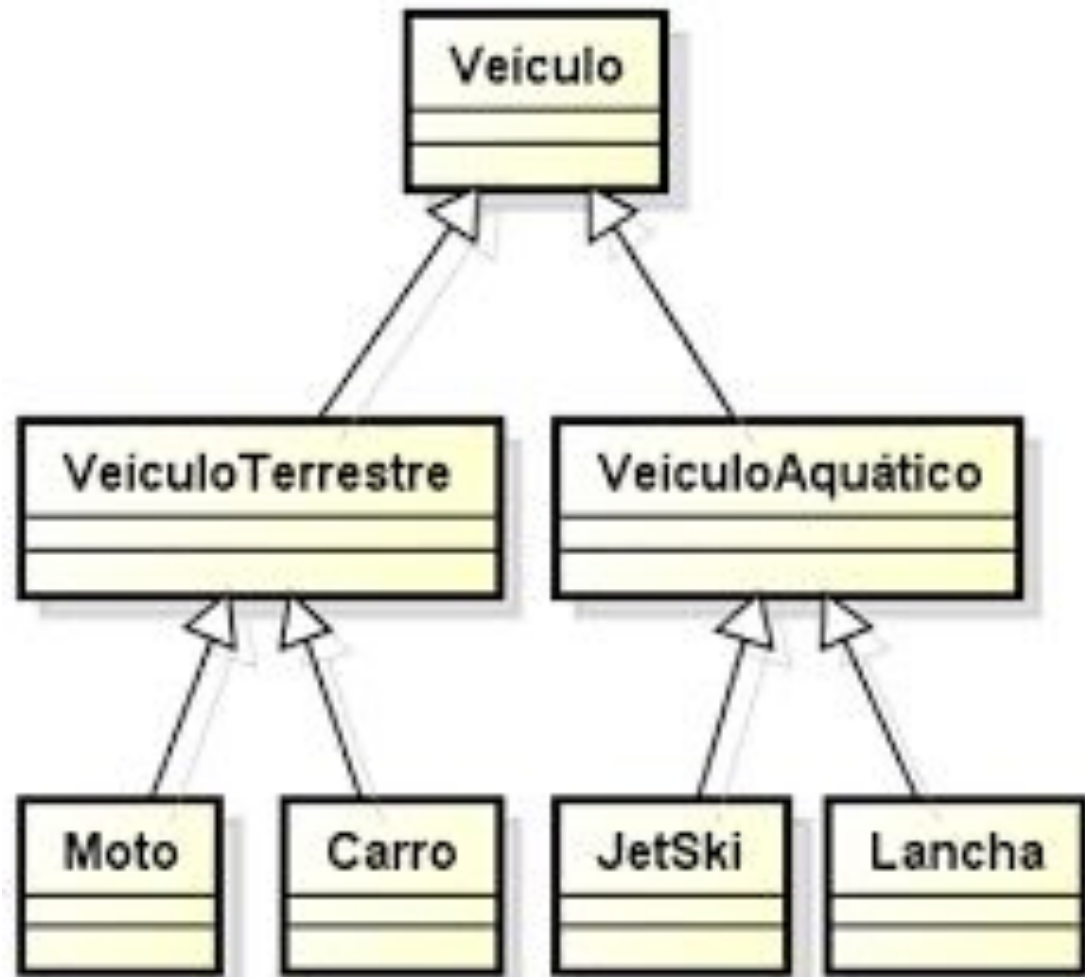
POO: abstração

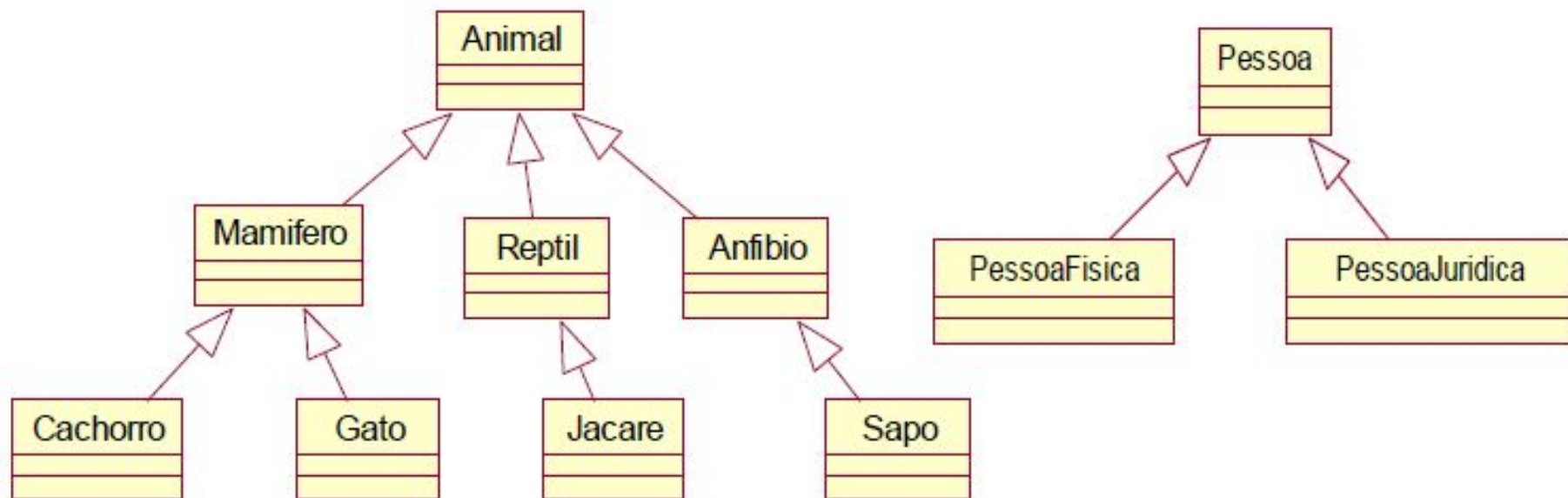
Abstração: Simplifica a complexidade ocultando detalhes desnecessários e expondo apenas as funcionalidades essenciais.

Moto

Carro







Importância da POO

A Programação Orientada a Objetos (POO) é fundamental no desenvolvimento de software moderno por várias razões, que são:

Importância da POO

Organização e Modularidade: A POO permite estruturar o código de maneira organizada e modular, tornando-o mais fácil de entender, manter e expandir.



Importância da POO

Reutilização de Código: Com conceitos como herança e polimorfismo, a POO promove a reutilização de código, o que reduz redundâncias e facilita a implementação de novas funcionalidades.



Importância da POO

Facilita a Manutenção: O encapsulamento permite isolar mudanças em partes específicas do código sem afetar outras, o que simplifica a manutenção e a correção de erros.



Importância da POO

Modelagem Natural de Problemas: A POO espelha o mundo real, permitindo que os desenvolvedores modelem problemas de forma intuitiva através de objetos que representam entidades e suas interações.

Importância da POO

Colaboração em Equipes: Com a POO, é mais fácil dividir o trabalho entre desenvolvedores, já que diferentes partes do sistema podem ser desenvolvidas e testadas independentemente.



Classe

Uma classe em Programação Orientada a Objetos (POO) é uma estrutura que serve como um **molde ou modelo** para criar objetos. Ela define um conjunto de **atributos** (dados) e **métodos** (funções) que os objetos criados a partir dessa classe irão possuir. Em outras palavras, a classe **especifica as características e comportamentos** comuns que seus objetos terão.

Classe

Por exemplo, em um programa que modela carros, uma classe **Carro** pode ter atributos como *cor*, *modelo*, *ano* e métodos como *acelerar()* e *frear()*. Cada objeto (ou instância) da classe Carro representará um carro específico, com suas próprias características e a capacidade de executar os comportamentos definidos.

Classe

Veiculo
-Cor -Chassi -placa
+EmitirGuiaSeguro()

Automovel
-numeroPlaca -cor -ano -tipoCombustivel -numeroPortas -quilometragem -renavam -numeroChassi -valorLocacao
+cadastrarAutomovel() +editarAutomovel() +removerAutomovel()

Carro
+id: string +ano : int +numero da placa : string +cor: string +combustivel: string +numero de portas: int +quilometragem: int +renavam: string +chassi: string +valor locacao: float
+debit()

Classe: Python

```
class Carro:
    def __init__(self, cor, modelo, ano):
        self.cor = cor          # Atributo de instância
        self.modelo = modelo    # Atributo de instância
        self.ano = ano          # Atributo de instância

    def acelerar(self):
        print(f"O {self.modelo} está acelerando.")

    def frear(self):
        print(f"O {self.modelo} está freando.")
```

Classe: Python (instanciação)

```
meu_carro = Carro("vermelho", "Ferrari", 2022)  
meu_carro.acelerar()  
meu_carro.frear()
```

Objeto

Objeto é uma **instância** de uma classe. Ele representa uma entidade concreta que possui **seus próprios atributos** (dados) e **métodos** (comportamentos), definidos pela classe da qual foi criado.

Objeto

Cada objeto pode ter valores diferentes para os atributos, mas compartilha os mesmos comportamentos definidos pela classe. Em termos simples, um objeto é o "resultado" ou "produto" da classe, que permite interagir e manipular as características e funcionalidades modeladas.

Objeto: exemplo

Por exemplo, em uma classe Carro, um objeto seria um carro específico, como uma Ferrari vermelha de 2022, com a capacidade de realizar ações como acelerar e frear.

```
meu_carro = Carro("vermelho", "Ferrari", 2022)
meu_carro.acelerar()
meu_carro.frear()
```

Objeto: exemplos

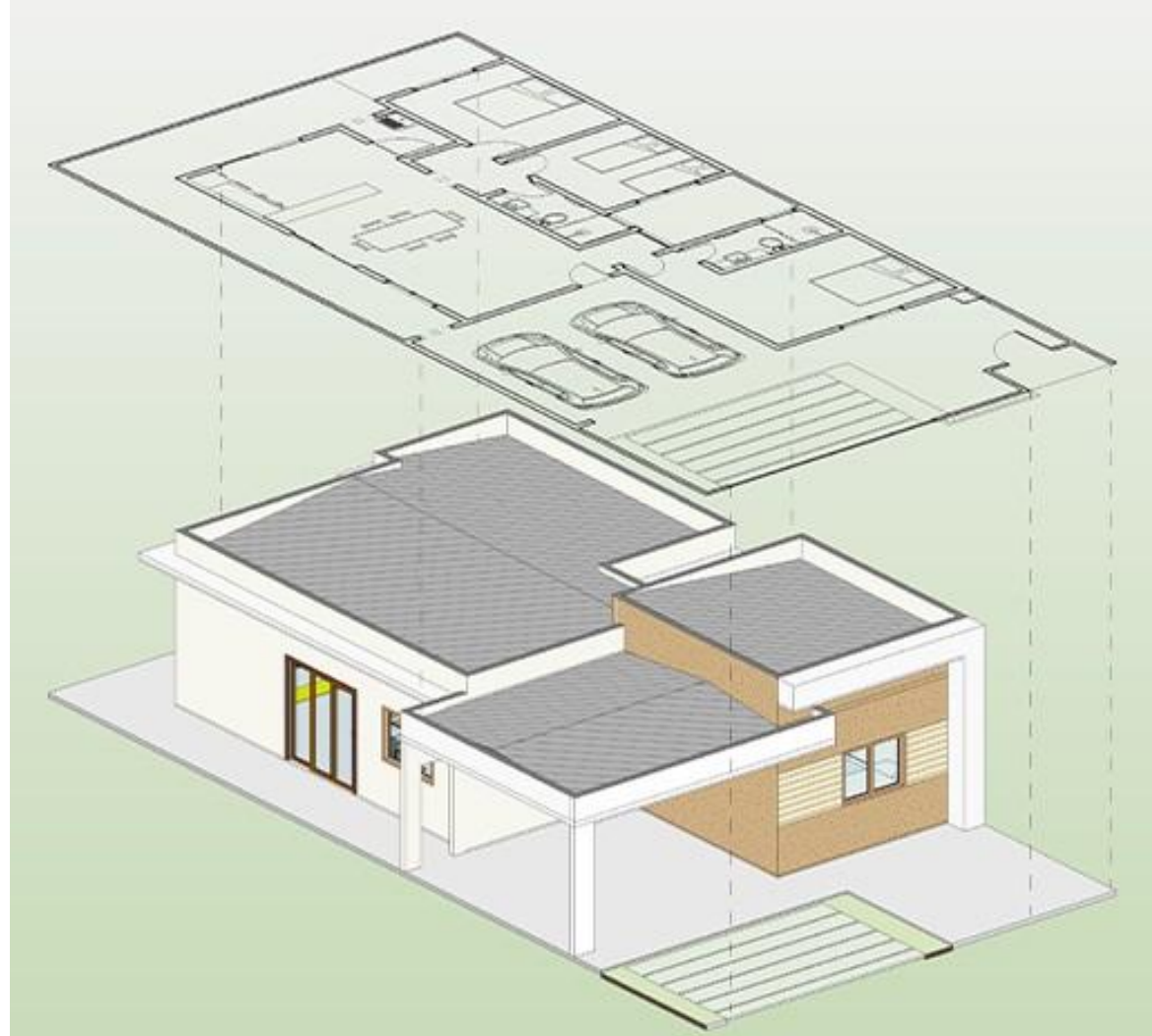
```
# Criando diferentes objetos da classe Carro  
carro1 = Carro("vermelho", "Ferrari", 2022)  
carro2 = Carro("azul", "Toyota", 2015)  
carro3 = Carro("preto", "BMW", 2020)
```

Classe vs Objeto

A classe é um modelo (projeto) que define a estrutura e o comportamento de objetos. Ela especifica quais atributos e métodos os objetos criados a partir dela terão.

Classe vs Objeto

Um objeto é uma instância concreta de uma classe. Ele é a manifestação real da classe, com valores específicos para seus atributos, permitindo que interaja com os métodos definidos pela classe.



Classe vs Objeto

Em resumo:

- **Classe:** Definição, estrutura e comportamento (teórico).
- **Objeto:** Instância concreta da classe (realização prática).

Atributos e Métodos

Atributos: São variáveis que armazenam dados ou propriedades de um objeto. Eles definem as **características** de um objeto, como cor, tamanho ou estado. Cada objeto pode ter valores específicos para seus atributos.

Atributos e Métodos

Métodos: São funções definidas dentro de uma classe que descrevem os **comportamentos** que os objetos podem executar. Eles operam sobre os atributos do objeto ou realizam ações relacionadas ao objeto.

Atributos e Métodos

Em resumo:

- **Atributos:** Definem o estado do objeto (dados).
- **Métodos:** Definem os comportamentos do objeto (ações).

Contato

@davimagals.dev

davimagal.sc@gmail.com